ПЛОВДИВСКИ УНИВЕРСИТЕТ „ ПАИСИЙ ХИЛЕНДАРСКИ“
ФАКУЛТЕТ ПО МАТЕМАТИКА И ИНФОРМАТИКА
КАТЕДРА СОФТУЕРНИ ТЕХНОЛОГИИ
ЛАБОРАТОРИЯ ПО ВЗАИМОДЕЙСТВИЕ ЧОВЕК-МАШИНА И СИМУЛАЦИИ

Физично-базирана симулация на полет по данни от автопилота на Mission Planner

ДИПЛОМНА РАБОТА

на

Иван Илиянов Станев

Специалност „Софтуерни Технологии и дизайн“

Факултетен номер: 2101681043

Научен ръководител: гл. ас. д-р Стефан Ставрев
Консултант: доц. д-р Стоян Аврамов, Телесис

Пловдив, 2025

PLOVDIV UNIVERSITY “PAISII HILENDARSKI”
FACULTY OF MATHEMATICS AND INFORMATICS
DEPARTMENT OF SOFTWARE TECHNOLOGIES
HUMAN-COMPUTER INTERACTION AND SIMULATIONS LABORATORY

A physics-based flight simulation using data from the Mission Planner autopilot

THESIS

by

Ivan Iliyanov Stanev

Major “Software technology and design”

Faculty number: 2101681043

Scientific supervisor: asst. prof. Stefan Stavrev
Consultant: assoc. prof. Stoyan Avramov, Telesys

Plovdiv, 2025

Съдържание

Списък на фигурите 4

Речник с използвана терминология и съкращения 5

Увод 7

Цели и задачи 8

Структура и обем 8

Глава 1. Автопилот, програмиране на летателна мисия и приемане на данни от дрона. 10

1.1 Автопилота Ardupilot 10

1.2 Наземен контрол. Mission Planner 11

1.2.1 Свързване на Mission Planner с ArduPilot 11

1.2.2 Планиране и изпълнение на мисия 13

1.3 Протоколът за пренос на данни MavLink 15

Глава 2. Протоколите за трансфер на данни TCP и UDP 19

2.1 Моделът OSI 19

2.2 Транспортен слой 20

2.3 Протоколът TCP 21

2.4 Протоколът UDP 23

2.5 Предимства и недостатъци на UDP и TCP 24

Глава 3. Unreal Engine 4 и софтуерната добавка Cesium 26

3.1.1 Обща характеристика 26

3.1.2 Blueprints 27

3.1.3 Нива в Unreal Engine 4 29

3.2 Софтуерната добавка Cesium for Unreal 30

3.2.1 Обща характеристика 30

3.2.2 Генериране на терен 31

3.2.3 Поставяне на обект в пространството 34

Глава 4. Реализация на приложението 37

4.1 Фаза на анализ и планиране 38

4.2 Фаза на проектиране 38

4.3 Фаза на разработване 39

4.3.1 Реализация на сървър с Python 40

4.4 Фаза на тестване 47

Заключение 48
Авторска справка за приносите 48
Перспективи за бъдещо развитие 49
Библиография 50

Списък на фигурите

Фигура 1: Изглед преди и след свързване на дрон към Mission Planner 12
Фигура 2: Прозорецът "Flight Paln" 13
Фигура 3: Автоматично генерирана мисия чрез вградената опция на Mission Planner 14
Фигура 4: Командата "LOITER_TURNS" добавя информация за броя на обиколките, посоката и местоположението, около което летателният апарат трябва да се задържи. 15
Фигура 5: MavLink v1. Структура на изпратеното съобщение 17
Фигура 6: MavLink v2. Структура на изпратеното съобщение 17
Фигура 7: OSI модел 20
Фигура 8: Адаптивно контекстно меню. Филтрира само съвместимите с възела с които работите в момента опции. 27
Фигура 9: Blueprint в който е реализирана следната функционалност: изстрелва играча във въздуха когато се докосне до друг обект 28
Фигура 10: Мениджър на съдържание с две добавени нива 30
Фигура 11: Генериран терен с реални мащаби и 3D съдържание с помощта на Cesium for Unreal 31
Фигура 12: Панелът за въвеждане на ключ за Cesium ion 32
Фигура 13: Прозореца Cesium 32
Фигура 14: Панел с детайли на Cesium терена 33
Фигура 15: Сгради, генерирани с помощта на Cesium 34
Фигура 16: Превръщане на обект в георефериран обект 35
Фигура 17: Летене на обект до предварително зададен маркер чрез функциите на Cesium 36
Фигура 18: Схема на потока на данни в разработката 39
Фигура 19: Тест при получаване на битове от Mission Planner да се показват на екрана 39

Фигура 20: Списък на използваните библиотеки 41
Фигура 21: Функцията която изпраща данните към симулатора 42
Фигура 22: Функцията която се грижи за получаването на данните 43
Фигура 23: Функцията main() 44
Фигура 24: Записване на пристигащата географска ширина в променлива 45
Фигура 25: Модел на дрона 45
Фигура 26: Структура на блупринта 46
Фигура 27: World outliner 46
Фигура 28: Визуализация на движение в пространството 47

Речник с използвана терминология и съкращения

В дипломната работа са използвани следните съкращения и термини:

- TCP - Transmission Control Protocol
- UDP - User Datagram Protocol
- Игрови енджин - вид софтуер за проектиране и създаване на компютърни игри.
- Софтуерна добавка - допълнително съдържание към софтуер, което добавя нови функционалности.
- UPS - United Parcel Service, компания за доставки на поща в Америка.
- DIYDrones- “do it yourself Drones ” любителски форум за дронове и всичко свързано с тях
- Роувър – Вид наземен дрон с колела обикновено приличащ на джип
- Хардуер- събирателно понятие за всички физически части на изчислително или друго устройство.
- Фърмуер- програми на вградени в устройства.
- GPS - Global Positioning System
- USB - Universal Serial Bus
- IP - Internet Protocol
- COM порт - communication port
- bps - Bytes per second

- UART - Universal Asynchronous Receiver-Transmitter
- APM - ArduPilot Mega – пълното наименование на конкретен автопилот
- EEPROM - Electrically Erasable Programmable Read-Only Memory
- WP - Waypoint
- XML - Extensible Markup Language
- MIT лиценз - Massachusetts Institute of Technology – лиценз за безплатно използване на определен код за научни цели
- CRC-16/MCRF4XX и CRC-16-CCITT – видове циклични кодове за проверка на данните по време на изпращане
- OSI - Open Systems Interconnection
- ISO - International Organization for Standardization
- IETF - Internet Engineering Task Force
- SMTP - Simple Mail Transfer Protocol
- DNS - Domain Name System
- DDoS - Distributed Denial of Service
- Контролни суми – битове, които се добавят към изпратените пакети, за да може да се провери дали данните, които са пристигнали, са верни или е възникнала грешка по време на изпращането им.
- WGS84 - World Geodetic System 1984
- ECEF - Earth-Centered, Earth-Fixed системата за задаване на ротация изпозвана от авто пилота Ardupilot
- Pitch, Yaw и Roll - трите основни измерения на движение или ротация на предмет в пространството
- ms - millisecond
- API - Application Programming Interface
- RTL – Return to launch

Увод

Темата на дипломната работа е подбрана, защото в нашето ежедневие все повече задачи започват да се изпълняват от дронове, независимо дали са управляеми или автономни. Дроновете могат да изпълняват разнообразие от задачи, за да улеснят живота на хората. Най-разпространено е използването им за целите на отбраната, като разузнаване, сканиране на терени и изпълнение на атаки. Пример за това е съвсем скорошната операция „Паяжина“, при която украински сили са използвали дронове, за да ударят ключови цели от ядрения запас на Русия [17]. От кадрите от атаката ясно се вижда, че за задаване на мисията е използван именно софтуер Mission Planner. Но те намират множество приложения и в цивилния сектор. Например, в земеделието, с тяхна помощ реколтата може лесно да бъде следена за ранно откриване на проблеми със засушаване или болести по растенията и да бъде прецизно пръскана с торове и пестициди. Друга област, в която дроновете могат да бъдат изключително полезни, е по време на търсене на хора, изчезнали в трудно достъпни терени, защото те могат да спестят ценно време, за локализиране изгубени хора, което може да доведе до тяхното спасяване. Напоследък време компании като Amazon и UPS вече тестват доставки с дронове, което може да ускори логистиката и да намали трафика.

Съществува разнообразие от различен софтуер за планиране и изпълнение на летателни мисии. Пример за това е AirSim, който е разработен от Microsoft и предоставя платформа с отворен код за симулация на дронове и автономни превозни средства с реалистична графика. Друг – QgroundControl, който също предоставя отворен код за наземния терминал и е съвместим с всички дронове, които използват протокола MavLink. Той е насочен основно към професионални потребители и предоставя софтуер, който не е толкова лесен за използване, но е много силен. Повечето от тях не наблягат на триизмерната визуализация и са значително сложни за използване особено за потребители, които ги използват за първи път.

Затова настоящата дипломна работа поставя следните цели с помощта на следните задачи.

Цели и задачи

Целта на настоящата дипломна работа е да се разработи симулатор на полети, като се подават данни от автопилот на софтуера за планиране на безпилотни полети Mission Planner. Основната идея е да се създаде опростена 3D среда, в която полетът на безпилотното летателно средство може да бъде проследява в реално време. Също така, тази среда ще дава възможност за добавяне на симулации на различни ситуации.

За постигане на целта са формулирани следните задачи:

- Да се изготви концепция за приложението която да подпомогне разработката
- Да се проучи и анализира софтуерът Mission Planner предоставен от създателите на ArduPilot
- Да се проучат и анализират протоколите за пренос на данни TCP, UDP и MavLink
- Да се направи анализ на игровия енджин Unreal Engine 4, както и на софтуерната добавка към него Cesium for Unreal
- Да се разработи модул, който отговаря за трансфера на данни от наземния контрол към симулатора
- Да се визуализират получените данни в Unreal Engine
- Да се разгледат резултатите и да се предложат начини за надграждане на разработения до момента софтуер

Структура и обем

Настоящата дипломна работа е с обем 51 страници и се състои от списък на фигурите, речник на използваната терминология и съкращения, увод, изложение в четири глави, заключение и библиография.

В първа глава се разглеждат автопилотът ArduoPilot, платформата за планиране и изпълнение на мисии с безпилотни летателни апарати Mission Planner и техния специализиран и оптимизиран протокол за пренос на данни от летателния апарат към наземния контрол MavLink.

Във втора глава са представени двата протокола за пренос на данни, UDP и TCP. Направена е съпоставка между двата протокола, представени са техните предимства и

недостатъци и е обяснено защо са подбрани и използвани при разработката на симулатора.

В трета глава се разглеждат основите на софтуера Unreal Engine 4. Разгледани са неговите компоненти, файловата структура на създадения проект. Подробно е разгледана и софтуерната добавка Cesium for Unreal, чрез която е изградено нивото за симулация на полета.

В последната, четвърта глава на дипломната работа, е описан подробно целият процес на софтуерната разработка. Представени са срещнатите проблеми в процеса на разработка и тяхното решаване.

В заключителната част от настоящата дипломна работа се обобщават постигнатите резултати и се набелязват перспективи за бъдещо развитие.

Приложената библиография е в обем от 2 страници и съдържа общо 16 литературни източника на английски език и 1 на български език. Тя обхваща 4 научни статии, 1 журналистическа статия и 12 интернет източника.

Глава 1. Автопилот, програмиране на летателна мисия и приемане на данни от дрона

Изпълнението на една летателна мисия включва значителна предварителна подготовка: настройване на автопилота, свързването му с наземния контрол, задаване на мисията и летателния план. В тази глава от дипломната работа подробно ще разгледаме автопилота ArduPilot [1], софтуера Mission Planner [2] и протокола MavLink [3], по който се извършва преносът на данни.

1.1 Автопилота Ardupilot

ArduPilot [1] започва своето развитие през 2007 г. като хоби проект с отворен код, създаден от Хорхе Сантана и Джейсон Шорт с цел изграждане на автопилотна система за малки безпилотни летателни апарати (дронове). Първоначално базиран на Arduino платформи, ArduPilot бързо привлича вниманието на общността с лесната си достъпност и гъвкавост. През следващите години проектът се разширява значително, включвайки поддръжка за самолети, хеликоптери, многороторни дронове и наземни и морски роботи. През 2011 г. е създадена общността DIY Drones [16], която става основен двигател за развитието на ArduPilot. До 2016 г. ArduPilot се превръща в една от най-напредналите и използвани отворени платформи за автономни системи, с богата функционалност, поддръжка на множество хардуерни платформи и широко приложение в научни, индустриални и хоби проекти. През 2016 г. става независим и до ден днешен се разраства благодарение на голямата общност около него.

ArduPilot позволява създаването и използването на надеждни, автономни, безпилотни системи за управление на превозни средства. ArduPilot предоставя цялостен набор от инструменти, подходящи за множество превозни средства и приложения. Като проект с отворен код, той непрекъснато се развива благодарение на бързата обратна връзка от голяма общност от потребители и разработчици. Екипът за разработка работи съвместно с общността и търговски партньори, за да добавя функционалности към ArduPilot, които са в полза на всички. Въпреки че ArduPilot не произвежда хардуер, неговият фърмуер работи с голямо разнообразие от различни хардуерни платформи за управление на автономни превозни средства от всякакъв тип. Примери за това са

мултикоптери, традиционни хеликоптери, самолети с неподвижно крило, лодки, подводници, роувъри и други. В съчетание със софтуер за наземен контрол, безпилотните превозни средства, работещи с ArduPilot, могат да разполагат с усъвършенствани функции.

Някои от тези функции, които този автопилот предоставя, са:

- Безпилотно изпълнение на предварително зададени мисии безпилотно
- Опции за аварийна защита в случай на загуба на сигнал, ниско ниво на батерията или други системни повреди.
- Триосово управление и стабилизация на камерата на дрона и връзка за предаване на живо с програмируем дисплей.
- Двупосочна комуникация в реално време между наземната станция и автопилота, включително GPS позиция, състояние на батерията и друга текуща информация.
- Пълно записване на данни за цялостен анализ след мисията, с инструменти за визуализация и картографиране чрез Google Earth.
- Безкрайни възможности за персонализация и разширени способности за мисии.
- Водещи в индустрията алгоритми за управление на всякакви видове автономни превозни средства.

1.2 Наземен контрол. Mission Planner

За изпълнението на произволна летателна мисия е необходим наземен контрол. Mission Planner [2] е безплотно приложение с отворен код, за наземен контрол за самолети, хеликоптери, мултикоптери и роувъри, разработено от Майкъл Оборне. Софтуерът е съвместим само с операционната система Windows. Може да се използва като инструмент за конфигуриране или като динамично допълнение за управление на автономно превозно средство.

1.2.1 Свързване на Mission Planner с ArduPilot.

За да е възможно изпълнението на дадена мисия, първо е необходимо Mission Planner да се свърже с автопилота. За да се установи тази връзка, първо се избира метод за комуникация, който ще се използва, и след това се настроива хардуера и драйверите

на устройства на Windows. Свързването към компютъра и автопилота може да се извърши чрез USB кабели, телеметрични радиомодули, Bluetooth, IP връзки и други методи. В Mission Planner връзката и скоростта на предаване на данни се настройват чрез падащите менюта в горната дясна част на екрана.

След като USB кабела или телеметричния радиомодул е свързан, Windows автоматично ще присвои COM порт номер на автопилота, който ще се покаже в падащото меню (самият номер няма значение). Също така, се задава подходящата скорост на предаване – обикновено 115200bps за USB връзка и 57600bps за радиовръзка. Чрез команди тази честота на предаване може да бъде сменена при необходимост. Най-малката която е налична е 1200 bps, а най-голямата е 1382400 bps. Но за нейното използване е необходим UART чип, който да може да обработва пристигащата с огромна скорост информация. В противен случай ще изгубим връзка с устройството. След като е избран желания порт и скорост на предаване на данни, се натиска бутона „CONNECT“, за да се установи връзка с автопилота. След установяване на връзката, Mission Planner ще изтегли параметрите от автопилота и бутонът ще се промени на „DISCONNECT“ (Фигура 1).



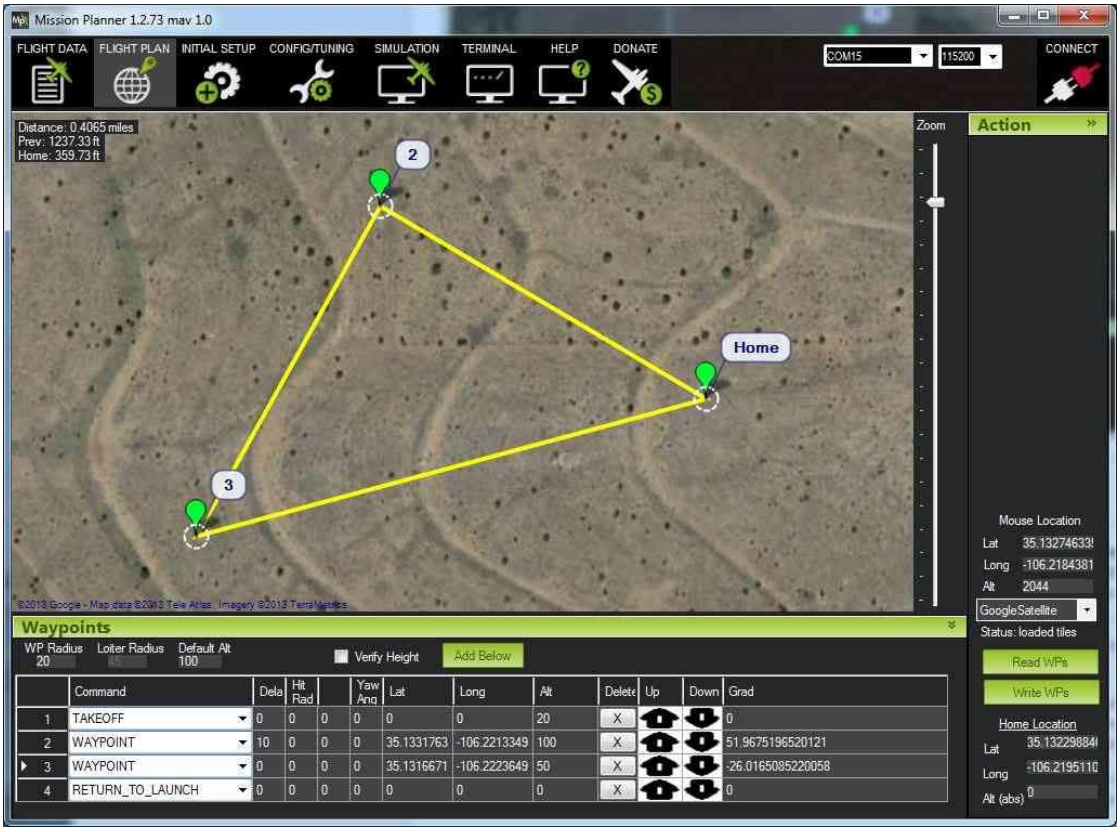
Фигура 1: Изглед преди и след свързване на дрон към Mission Planner

Mission Planner предоставя и опция за свързване на повече от едно превозно средство едновременно. Може да се свърже второ превозно средство, като се натисне десен бутон на мишката върху бутона "DISCONNECT", или ако му се предостави предварително изготвен списък с различните портове, с които искаме да се свържем, форматиран по следния начин:

- tcp://127.0.0.1:5670
- udp://127.0.0.1:14550
- udpcl://192.168.1.255:14550
- serial:com4:115200

1.2.2 Планиране и изпълнение на мисия

След като дронът е успешно свързан е време да се планира мисията. За планиране на мисии се използва менюто „Flight Plan“ в Mission Planner (Фигура 2). Първо трябва да се зададе начална точка на мисията „Home location“. От тази точка ще се активира дронът, и там той ще се върне след приключване на мисията, освен ако изрично не е зададена друга крайна точка или команда, която противоречи на връщането му.



Фигура 2: Прозорецът "Flight Paln"

Могат да се въвеждат точки по маршрута (waypoints) и други команди. В падащите менюта на всеки ред се избира желаната команда. Заглавията на колоните ще се променят, за да покажат какви данни се изискват за тази команда. Географска ширина и дължина могат да се въведат чрез кликване върху картата. Височината е относителна спрямо началната позиция на дрона, така че, ако зададем 100 метра, дронът ще лети 100 метра над нас. По подразбиране, височината е зададена при въвеждане на нови точки по маршрута. Ако изберем опцията „Verify height“, Mission Planner ще използва топографски данни от Google Earth, за да коригира зададената от нас височина според релефа под всяка точка. Така, ако дадена точка се намира на хълм, Mission Planner ще

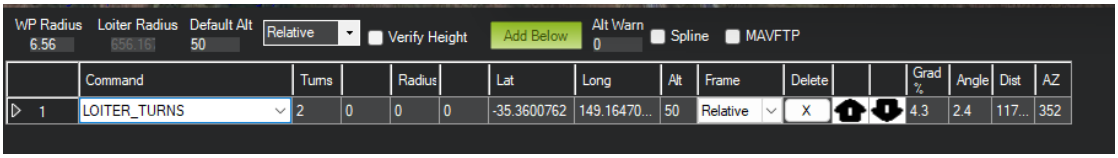
увеличи ALT стойността с височината на хълма. Това е добър начин да се предпази дрона от сблъсъци с терена! След като мисията приключи, изберете бутона „Write“, за да я изпратите към APM и да я запишете в EEPROM паметта. Този вид памет е енергонезависима, препрограмируема с електрически запис и електрическо изтриване и се използва в компютрите за съхранение на малки обеми от данни. Може да се потвърди, че всичко е наред, като се натисне бутонът „Read“. Могат да се запазят няколко файла с мисии локално чрез „Save WP File“ или да се зареди мисия чрез „Load WP File“ от менюто с десен клик.

Mission Planner предоставя и опцията да създаде автоматично мисия (Фигура 3), което е особено полезно, когато летателният апарат трябва да лети в така наречения „lawnmower pattern", в който той лети напред и назад, за да събира снимки най-често използвани за картографиране. За да се картографира добре една област, дронът трябва да я заснеме от няколко различни ъгъла, след което да се съпоставят заснетите резултати и да се представи полученият терен. За да се постигне това, мисията се програмира така, че на всяка успоредна ивица дронът заснема и част от предходната. След завършване на цялата мисия с данни от различни ъгли може да бъде пресметнат правилно релефът на картографирания терен картографиран.



Фигура 3: Автоматично генерирана мисия чрез вградената опция на Mission Planner

Mission Planner предоставя подбран списък с команди, подходящ за текущия тип превозно средство, и добавя заглавия на колони за параметрите, изискващи стойности от потребителя. Това включва навигационни команди за придвижване до точки по маршрута и задържане в даден район, "DO" команди за изпълнение на конкретни действия (например заснемане на снимки), както и команди с условие, които определят кога дадени "DO" команди (Фигура 4) могат да бъдат изпълнени.



Фигура 4: Командата "LOITER_TURNS" добавя информация за броя на обиколките, посоката и местоположението, около което летателният апарат трябва да се задържи.

Броят на различните елементи на мисията, които могат да се съхраняват в автопилота, е ограничен до приблизително 650 точки от маршрута, или около 10kB. Ако е необходимо да се изпълни по-дълга мисия, може да се зададе параметърът BRD_SD_MISSION, който създава файл на SD картата за съхранение на допълнителни данни. Този параметър може да бъде настроен до максимум 64 kB, което позволява общо над 4000 точки от маршрута в една мисия.

Когато списъкът с елементи на мисия приключи, мултикоптерът ще задържи позицията си (loiter) на последната точка по маршрута безкрайно, докато не настъпи смяна на режим или изключване, например при кацане. Роувърът ще спре и ще задържи позицията си, но самолетът ще изпълни връщане към началната точка (RTL), освен ако последният елемент от мисията не е безкрайно задържане (unlimited loiter), завъртане (turns) или кацане.

1.3 Протоколът за пренос на данни MavLink

За комуникацията между дроновете и наземния контрол се използва протоколът MAVLink. Това е изключително лек комуникационен протокол за обмен на съобщения с дронове и между отделните компоненти на борда на дрона.

MAVLink [3] използва модерен хибриден модел на публикуване–абониране и директна комуникация точка-точка. Потоците от данни се изпращат като теми (topic), докато конфигурационните подпротоколи, като протоколът за мисии или

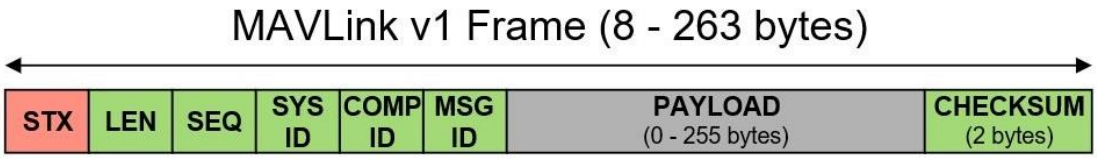
параметричният протокол, са от тип точка към точка с възможност за повторно предаване.

Съобщенията се дефинират в XML файлове. Всеки XML файл дефинира набора от съобщения, които се поддържат от конкретна MAVLink система, наричана още "диалект". Референтният набор от съобщения, който се използва от повечето наземни станции и автопилоти, е дефиниран във файла common.xml. Повечето диалекти надграждат тази дефиниция. Генератори на код създават софтуерни библиотеки за конкретни програмни езици от тези XML дефиниции на съобщенията. Тези библиотеки могат да се използват от дронове, наземни контролни станции и други MAVLink-съвместими системи за комуникация. Генерираните библиотеки обикновено са с MIT лиценз, което означава, че могат да се използват без ограничения, включително в затворен код, без задължение за публикуване на изходния код на приложението.

Протоколът е бинарен телеметричен, създаден за системи с ограничени ресурси и връзки с ограничена пропускателна способност, тъй като е разработен специално за използване за комуникация между дронове. Това означава, че ресурсите са ограничени и връзката може да не е стабилна. MAVLink се използва в две основни версии: v1.0 и v2.0, като v2.0 е обратно съвместима и може да приема и изпраща пакети във формат v1.0.

Съобщенията, изпратени от версия едно (Фигура 5), имат дължина от 8 до 263 байта и съдържат следните полета:

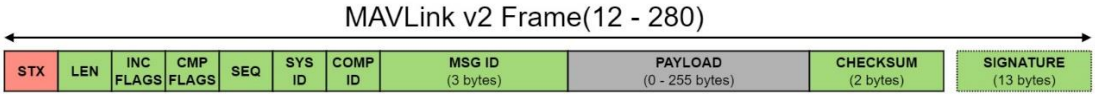
- STX – Стартов байт, указващ началото на съобщението (обикновено 0xFE).
- LEN – Дължина на полезния товар (payload) в байтове.
- SEQ – Номер на последователността, използван за проследяване на изгубени съобщения.
- SYS ID – Идентификатор на системата (напр. дрон или наземна станция).
- COMP ID – Идентификатор на компонента (напр. автопилот, камера).
- MSG ID – Идентификатор на типа съобщение.
- PAYLOAD – Данните, съдържащи информацията от съобщението (до 255 байта).
- CHECKSUM – Контролна сума за проверка на целостта на съобщението.



Фигура 5: MavLink v1. Структура на изпратеното съобщение

Съобщенията, изпратени от версия две (Фигура 6), имат дължина от 12 до 280 байта и съдържат следните полета:

- STX – Стартов байт (0xFD), различаващ се от v1.
- LEN – Дължина на полезния товар.
- INC FLAGS – Индикатори за допълнителни опции като криптиране или подпис.
- CMP FLAGS – Флагове за компресиране на съобщението.
- SEQ – Номер на последователността.
- SYS ID – Идентификатор на системата.
- COMP ID – Идентификатор на компонента.
- MSG ID – Разширен идентификатор на съобщението (3 байта вместо 1).
- PAYLOAD – Основните данни на съобщението.
- CHECKSUM – Контролна сума.
- SIGNATURE (по избор) – Цифров подпис за удостоверяване и защита на съобщението (13 байта, ако е активиран).



Фигура 6: MavLink v2. Структура на изпратеното съобщение

Телеметричните потоци от данни се изпращат чрез мултикаст архитектура, докато аспекти на протокола, които променят конфигурацията на системата и изискват гарантирана доставка, като протоколът за мисии или параметричният протокол, използват комуникация точка до точка с възможност за повторно предаване.

MAVLink прилага два механизма за проверка на целостта на данните. Първата проверка се извършва върху целостта на пакета по време на предаване и използва контролна сума CRC-16/MCRF4XX. Тази проверка гарантира, че данните не са

променени по време на предаването, но не осигурява съответствие със структурата на съобщението. Втората проверка се извършва върху описанието на данните, за да се гарантира, че две съобщения с един и същи идентификационни кодове съдържат наистина еднаква информация. За да се постигне това, дефиницията на съобщението се подлага на CRC-16-CCITT, а получената стойност се използва за инициализиране на контролната сума на пакета. В повечето референтни реализации тази стойност се съхранява в масив, наречен „CRC_EXTRA“.

Глава 2. Протоколите за трансфер на данни TCP и UDP

В настоящата глава ще бъдат разгледани двата най-често използвани транспортни протокола. Ще бъдат разгледани OSI моделът и транспортният слой, който е част от него. Ще бъде представено сравнение между тях. Ще бъдат подчертани техните предимства и недостатъци. Читателят ще добие представа защо е избрано използването на всеки от дадените протоколи в тази разработка.

2.1 Моделът OSI

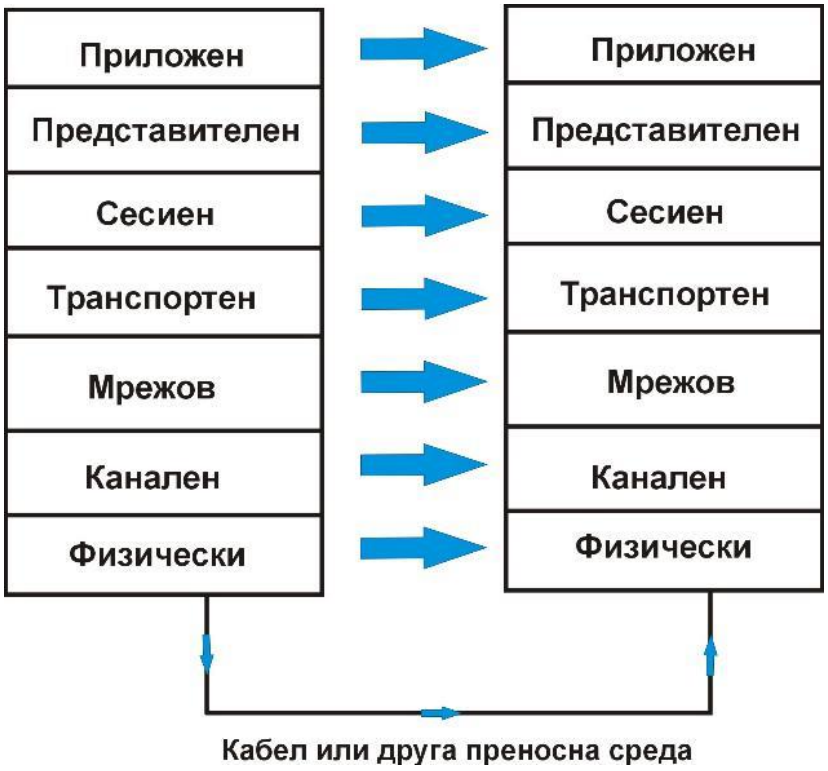
И двата транспортни протокола, които се разглеждат в тази глава от настоящата дипломна работа, са част от транспортния слой на модела OSI. За да сме напълно запознати с тях, ще е добре първо да се запознаем и със самия модел.

Моделът, създаден от Международната организация по стандартизация (ISO), позволява на различни комуникационни системи да комуникират помежду си чрез стандартни протоколи.

OSI моделът (Фигура 7) може да се разглежда като универсален език за компютърни мрежи. Той се базира на концепцията за разделяне на комуникационната система на седем абстрактни слоя, всеки от които надгражда предходния. Тези слоеве са:

- Физически (Physical) - отговаря за физическото предаване на данни чрез кабели, електрически сигнали или радиовълни.
- Канален (Data Link) - осигурява надеждна връзка между два устройства и открива и може да коригира грешки при предаване.
- Мрежов (Network) - управлява адресирането и маршрутизирането на данни между различни мрежи.
- Транспортен (Transport) - гарантира, че данните се доставят точно, надеждно и в правилния ред.
- Сесиен (Session) - установява, управлява и прекратява комуникационни сесии между приложения.

- Представителен (Presentation) - форматира и кодира данните така, че различните системи да ги разбират, например криптиране или компресия.
- Приложен (Application) - осигурява интерфейс между крайните потребители и мрежата чрез приложения като уеб браузъри и имейл клиенти.



Фигура 7: OSI модел

Всеки слой в OSI модела изпълнява конкретна функция и комуникира със слоевете над и под него. Всеки от слоевете на изпращача може да комуникира само със съответния слой на получателя.

2.2 Транспортен слой

Транспортният слой отговаря за комуникацията от край до край между две устройства. Това включва вземане на данни от сесийния слой и разбиването им на части, наречени сегменти, преди да ги изпрати към мрежовия слой. Транспортният слой на приемащото устройство е отговорен за повторното сглобяване на сегментите в данни, които сесийният слой може да използва.

Транспортният слой също така отговаря за контрол на потока и контрол на грешки. Контролът на потока определя оптимална скорост на предаване, за да гарантира,

че изпращач с бърза връзка няма да претовари получател с бавна връзка. Контролът на грешки се извършва на приемащия край чрез проверка на пълнотата на получените данни, и ако те не са пълни, се заявява повторно изпращане.

Протоколите, използвани в транспортния слой, включват Transmission Control Protocol (TCP) и User Datagram Protocol (UDP).

2.3 Протоколът TCP

Протоколът за управление на предаването (TCP)[13] е комуникационен стандарт, който позволява на приложения и устройства да обменят съобщения. Той е проектиран да изпраща пакети през интернет, като гарантира успешното доставяне на данните.

TCP е включен в стандартите, дефинирани от Internet Engineering Task Force (IETF). Той е един от най-често използваните протоколи в цифровите мрежови комуникации и осигурява доставяне на данни от край до край.

TCP организира данните така, че да могат да се предават между сървър и клиент. Преди започване на предаването, TCP установява връзка между изпращача и получателя, като се уверява, че тя остава активна до изпращането на първото съобщение. След това разделя големите количества данни на по-малки пакети, като поддържа целостта на данните през целия процес.

User Datagram Protocol (UDP) е алтернатива на TCP в мрежите, на която е посветена следващата част от дипломната работа.

TCP най-често работи съвместно с протокола IP. IP получава и определя адреса на приложението или устройството, до което трябва да бъдат изпратени данните. След това TCP отговаря за транспортирането и маршрутизирането на данните през мрежовата инфраструктура и гарантира, че те ще бъдат доставени точно до приложението или устройството, посочено от IP адреса. Работейки заедно, тези две технологии позволяват комуникация между устройства на големи разстояния и осигуряват възможно най-ефективно предаване на данни до местоназначението им. Ето защо процесът обикновено се нарича просто TCP/IP. При наличието на подходящи мерки за сигурност, комбинацията от TCP и IP позволява безопасно и надеждно прехвърляне на данни между две или повече устройства.

Съобщенията се разделят на пакети, за да се избегне повторното изпращане на цялото съобщение в случай на проблем по време на предаването им. Пакетите се сглобяват автоматично, след като достигнат своето местоназначение. Всеки пакет може да поеме по различен маршрут от изпращача до получателя в в зависимост от първоначалният маршрут е претоварен или е недостъпен.

TCP/IP разделя комуникационните задачи на слоеве, което поддържа процеса стандартизиран, така че да се използва от всички хардуерни и софтуерни производители. Данните трябва да преминат през четири слоя, преди да достигнат до получателя. След това TCP/IP преминава през слоевете в обратен ред, за да възстанови съобщението в първоначалния му вид.

TCP установява и поддържа връзка между приложения или устройства, докато обменят данни. Той определя как съобщението да се раздели на пакети, номерира и сглобява пакетите и ги изпраща към други устройства в мрежата, като рутери, защитни стени и комутатори, и накрая към крайния получател. TCP също изпраща и получава пакети от мрежовия слой, управлява предаването на загубени пакети, контролира потока на данни и гарантира, че всички пакети достигат до местоназначението си.

Добър пример за това как работи този процес на практика е изпращането на имейл чрез SMTP от имейл сървър. За да започне процесът, TCP слоят на сървъра разделя съобщението на пакети, номерира ги и ги препраща към IP слоя, който транспортира всеки пакет до сървъра на получателя. Когато пакетите пристигнат, те се предават на TCP слоя, който ги сглобява в оригиналния формат на съобщението и го предава на имейл сървъра, за да бъде доставени в пощенската кутия на потребителя.

TCP/IP използва така нареченото тристепенно ръкостискане, за да установи връзка между устройство и сървър, което гарантира, че множество TCP сокет връзки могат да се предават едновременно двупосочно. И двете страни – устройството и сървърът – трябва да синхронизират и потвърдят получаването на пакетите, преди да започне комуникацията. След което могат да договорят, да разделят и да прехвърлят TCP сокет връзките.

2.4 Протоколът UDP

User Datagram Protocol (UDP)[14] е комуникационен протокол, предназначен за приложения, при който е важно данните да се предават бързо, като видеоигри, видеоклипове или заявки към Domain Name System (DNS). UDP осигурява по-бърза комуникация, тъй като не отделя време за създаване на стабилна връзка с получателя, преди да предаде данните. Понеже установяването на връзка отнема време, пропускането на тази стъпка води до по-висока скорост на предаване на данните, но това може да доведе до други проблеми, свързани със сигурността и цялостта на данните. Един от тези проблеми при използването на UDP е възможността за загуба на пакети по пътя от изпращача до получателя. Освен това протоколът улеснява злонамерени действия, като разпределени атаки за отказ на услуга (DDoS).

В много случаи, особено при Transmission Control Protocol (TCP), когато данните се предават през интернет, те не само трябва да бъдат изпратени, но и получателят трябва да сигнализира, че е готов да ги приеме. Едва когато и двете страни са готови, предаването може да започне. При UDP обаче данните се изпращат, преди да е установена стабилна връзка. Това може да доведе до проблеми при прехвърлянето на данни, а също така създава възможност за хакери, които целят да извършат DDoS атаки. Тези атаки представляват изпращане на голям брой UDP заявки едновременно, за да претоварят получателя с тяхната обработка и да не позволят да се обработват други заявки.

В сравнение с други мрежови протоколи, процесът на UDP е доста прост. Целевият компютър се идентифицира и към него се изпращат пакети с данни, наречени "датаграми". Няма механизъм, който да указва реда, в който трябва да пристигнат пакетите. Също така няма процес за проверка дали датаграмите изобщо са достигнали до дестинацията си.

Макар че UDP използва контролни суми (checksums) за проверка на целостта на данните и портови номера за разграничаване ролята на данните при изпращача и получателя, липсата на задължително ръкостискане създава проблем. Програмата, която потребителят използва с помощта на UDP, остава изложена на ненадеждните аспекти на подлежащата мрежова инфраструктура. В резултат на това данните може да бъдат

доставени, а може и да не бъдат доставени. Редът на пристигане не се контролира, както е при TCP, така че начинът, по който се появяват в крайна сметка, може да бъде проблематичен, например, да се покажат със закъснение, в грешен ред или с липсващи части.

Въпреки това, в ситуации, в които няма нужда от проверка за грешки или корекция на данните, това не представлява сериозен проблем. Това е една от причините UDP да се използва във видео приложения – по-важно е видеосигналът да достигне навреме, дори ако понякога има смущения или влошаване в качеството.

2.5 Предимства и недостатъци на UDP и TCP

За да бъдат пренесени каквито и да е данни в интернет, към тях се добавя заглавна част със служебна информация, като адрес на подателя, адрес на получателя, дължина на изпратеното съобщение, кодове за проверки на грешки, номериране на отделни части на съобщението и много други. Тази заглавна част в протокола UDP е фиксирана на само 8 байта и е сравнително проста и включва:

- 16-битов порт на изпращача
- 16-битов порт на получателя
- 16-битова които указват дължината на съобщението
- 16-битова контролна сума (checksum)

Това е цялата информация, използвана за изпращане на датаграма до нейната дестинация. Процесът на изпращане не включва проверка за установена връзка между източника и получателя.

При TCP обаче заглавната част може да варира от 20 до 60 байта, което позволява изпращането на повече проверки за целостта и верността на данните при предаването им. Освен това, TCP изисква "ръкостискане" между изпращача и получателя. Това прави TCP по-надежден от UDP. По време на TCP комуникация данните могат да бъдат изпратени едва след като връзката между двете страни е официално установена. При UDP, тъй като връзка не е необходима, данните могат да се изпращат веднага.

Друга разлика между TCP и UDP е, че при TCP редът на получаване на пакетите се установява още преди да започне прехвърлянето. TCP също така осигурява

потвърждение, че пакетите са пристигнали както трябва. Ако някой пакет не пристигне, TCP изисква той да бъде изпратен отново. UDP не изисква потвърждение, проверка или повторно изпращане. Това осигурява целостта на данните, когато те се изпращат чрез TCP, което е едно от основните предимства на този протокол.

Ако дадено приложение използва UDP, потребителите поемат риска от грешки, загуба на данни, липса на доставка или дублиране. Ползата от този компромис е по-високата скорост. Самият UDP не е задължително отговорен за загубата на данни. Информацията в хедъра е достатъчна, за да стигнат датаграмите до целта, а хронологичният ред на изпращане по принцип трябва да ги поддържа в правилния ред. Въпреки това, повечето мрежови рутери не могат да потвърдят получаването или да поддържат реда на пакетите. Пакетите с данни могат да бъдат загубени или дублирани. Въпреки този недостатък, UDP компенсира с високата скорост на предаване. Което е и основното му предимство.

Поради тези огромни разлики между двата транспортни протокола, те са се утвърдили като най-използваните. Те покриват различни нужди на потребителите. Но е изключително важно да бъде добре подбран начинът им на използване, за да могат да се компенсират техните недостатъци и да се подчертаят техните предимства. Ако информацията, която изпращаме е чувствителна и е необходимо да няма загуба на данни, например при плащания, е уместно да изберем TCP. Понеже той гарантира, че нашите данни ще пристигнат точно във вида, в който са изпратени. Но ако скоростта е важна, например при излъчване на спортно събитие на живо ще е уместно да използваме UDP, за да няма излишно закъснение, дори и ако има известна загуба на качеството.

Глава 3. Unreal Engine 4 и софтуерната добавка Cesium

Третата глава на настоящата дипломна работа ще запознае читателя с игровия енджин Unreal Engine 4 и със софтуерната добавка към него Cesium, която позволява лесно създаване на игрово ниво от реалния свят с помощта на сателитни изображения, предоставени от Google Maps или Bing.3.1 Unreal Engine 4

3.1.1 Обща характеристика

Unreal Engine 4 е мощен енджин, създаден от Epic Games и използван за проектиране и разработване на видеоигри. Той предоставя на разработчиците пълен набор от инструменти и технологии за създаване на интерактивна 3D графика и впечатляващи виртуални преживявания. Енджинът се разработва вече повече от две десетилетия и се е превърнал в гъвкава платформа, предоставяща множество функции.

Все повече разработчици избират Unreal Engine 4 заради неговите възможности, производителност и визуалното качество, което той предоставя. Енджинът предлага необходимите инструменти и работни процеси за създаване на висококачествени игри с реалистична графика. Някои от по-важните функции включват динамично осветление със сенки и подвижни светлинни източници, както и усъвършенствани физични симулации, позволяващи симулация на течности, физика на коса и други.

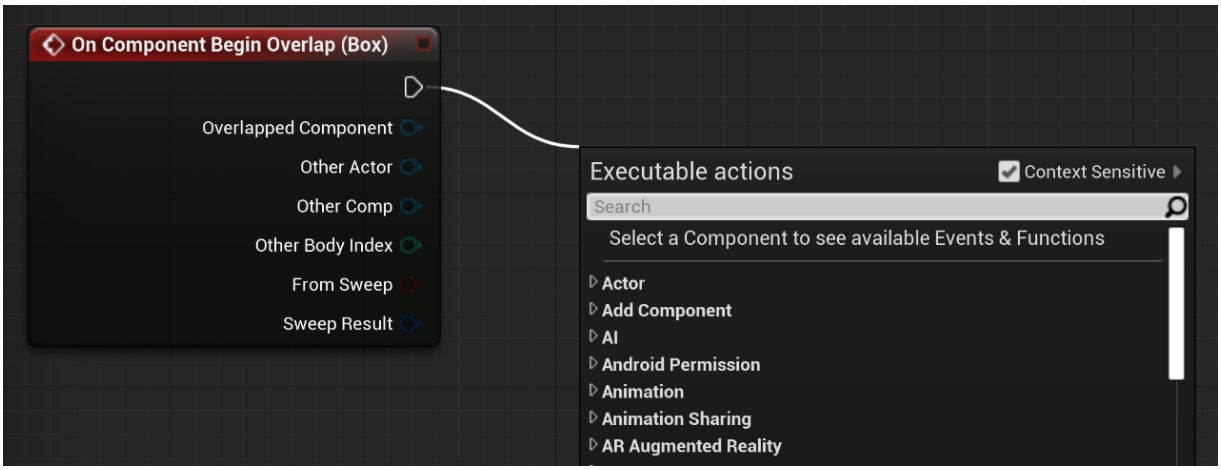
Unreal Engine 4 е изграден върху модулна архитектура, която позволява на разработчиците да поддържат кода подреден и разделен на компоненти. Това позволява лесно да обновяват или добавят ново съдържание, без да нарушават работата на играта. Unreal Engine 4 предлага също така богат набор от визуални ефекти и инструменти за рендиране, както и широка гама от допълнителни функции, като симулации на физика, инструменти за анимация, аудио системи, мрежова функционалност и мощната система за визуално програмиране Blueprints, която е достъпна и за хора с ограничен опит в програмирането.

Благодарение на тези възможности, Unreal Engine 4 намира приложение не само в разработката на игри, но и в архитектурната визуализация, симулации, виртуална

реалност, музикални видеа и анимационни филми. Алтернативен, но не толкова добър игрови енджин, е Unity. Но за целите на тази разработка, е подбран Unreal Engine 4.

3.1.2 Blueprints

Blueprint Visual Scripting е визуален език за програмиране, който използва интерфейс с възли (nodes) (Фигура 8) за създаване на елементи на геймплея. Тази работна среда предоставя на дизайнерите широк набор от скриптови концепции и инструменти, които обикновено са достъпни само за програмисти. Освен това, специален Blueprint markup, достъпен в C++ имплементацията на Unreal Engine, позволява на програмистите да създават базови системи, които дизайнерите могат да разширяват чрез Blueprints[5].



Фигура 8: Адаптивно контекстно меню. Филтрира само съвместимите с възела с които работите в момента опции.

Подобно на много скриптови езици, системата Blueprint позволява дефинирането на обектно-ориентирани класове и обекти в енджина. Системата и създадените чрез нея обекти обикновено се наричат просто "Blueprints".

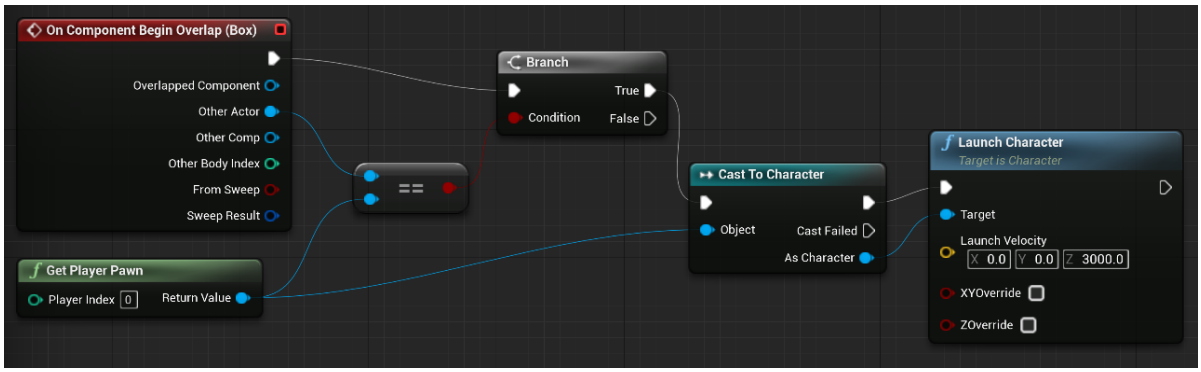
UnrealScript е оригиналният скриптов език на Unreal Engine, използван за програмиране на геймплей събития и логика преди появата на Unreal Engine 4. Езикът е създаден от Тим Суини и е обектно-ориентиран, подобно на Java, но без множествено наследяване и методи с еднакви имена. С появата на Unreal Engine 4 (март 2014 г.) UnrealScript беше заменен от C++ и визуалното скриптиране с Blueprints.

Blueprints не са директен заместител на UnrealScript, но изпълняват много от същите функции, включително:

- Разширяване на класове

- Съхраняване и промяна на стойности по подразбиране
- Управление на инстанции на под-обекти (например Components) в класовете

Очакванията са, че програмистите ще създават базови класове, които предоставят функции и свойства, достъпни и разширяеми чрез Blueprints, направени на тяхна база (Фигура 9).



Фигура 9: Blueprint в който е реализирана следната функционалност: изстрелва играча във въздуха когато се докосне до друг обект

Blueprint-ите могат да бъдат от няколко различни типа, всеки от които има своето специфично предназначение. От създаване на нови типове обекти, през скриптиране на събития в нивото, до дефиниране на интерфейси, които се използват от други Blueprint-и. Ето някои от тези видове.

3.1.2.1 Blueprint Class

Класът Blueprint, често наричан просто Blueprint, позволява на създателите на съдържание лесно да добавят функционалност към съществуващи класове за геймплей. Blueprint-ите се създават визуално в Unreal Editor, вместо чрез писане на код, и се запазват в content package. По същество те дефинират нов клас или тип Actor, който след това може да бъде поставен в нивата като инстанции, и се държат като всеки друг тип Actor.

3.1.2.2 Data-Only Blueprint

Data-Only Blueprint е Blueprint клас, който съдържа само код, променливи и компоненти, наследени от своя родител. Този тип Blueprint позволява модифициране на наследени свойства, но не могат да се добавят нови елементи. По същество, те са заместители на архетипи и се използват, за да дадат възможност на дизайнерите да настройват свойства или да създават варианти на обекти.

Data-Only Blueprints се редактират чрез компактен редактор на свойства, но могат лесно да бъдат преобразувани в Blueprints, като просто се добавят код, променливи или компоненти чрез Blueprint Editor.

3.1.2.3 Blueprint Interface

Blueprint Interface е колекция от една или повече функции (само с имена, без реализация), които могат да бъдат добавени към Blueprint-и, които ги наследяват. Всеки Blueprint, към който е добавен интерфейсът, гарантирано съдържа тези функции. Функциите могат да получат различна реализация във всеки Blueprint, който ги е наследил. Това е аналогично на интерфейсите в обектно-ориентираното програмиране – общ начин различни обекти да комуникират чрез еднакъв интерфейс.

Blueprint Interface може да бъде създаден подобно на други Blueprint-и, но има някои ограничения:

- Не могат да бъдат добавени нови променливи
- Не могат да бъдат редактирани графи
- Не могат да бъдат добавени компоненти

3.1.3 Нива в Unreal Engine 4

Нивото [4] е цялото или част от „света“ на играта. Нивата съдържат всичко, което играчът може да види, с което може да взаимодейства, като среди, обекти, други герои и т.н. Във видеоигрите е обичайно да има множество нива с ясно разграничени преходи между тях. Например, след като победите последния бос в дадено ниво, преминавате към следващото ниво. За други типове интерактивни преживявания, създадени с Unreal Engine 4, можете да използвате различни нива (Фигура 10), за да преминавате между различни видове среди или демонстрации.

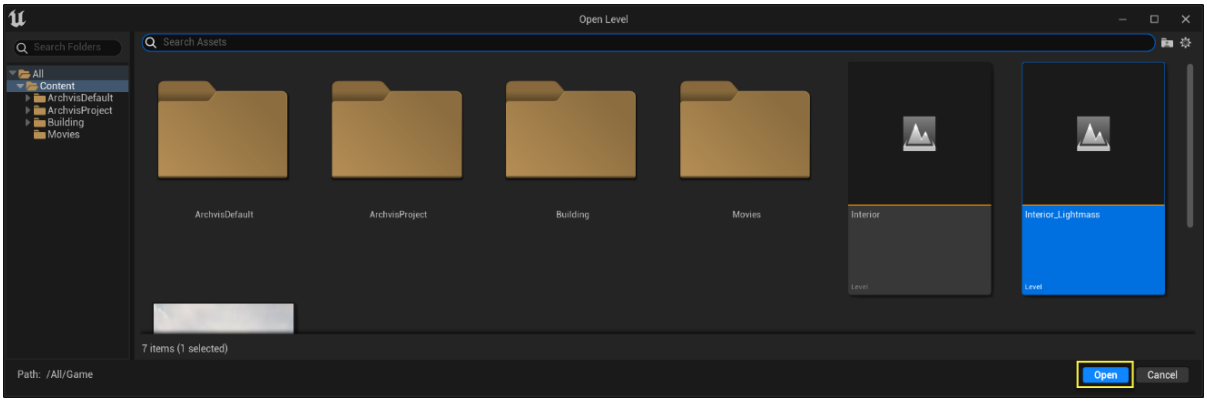
Unreal Engine 4 запазва всяко ниво като отделен .umap файл, поради което нивата понякога се наричат „картички“.

Едно ниво обикновено съдържа тези елементи:

- .umap файлът – самото ниво. Контейнерът, който съдържа всичко останало.

- Среда, изградена от Static Mesh актьори, включва дървета, скали, стени и други елементи на околната среда. Някои среди използват и други типове актьори, като актьори Landscape или Water.
- Играчът е представен чрез Skeletal Mesh актьор.
- Един или повече източника на светлина от различни типове.
- Фонови и звукови ефекти (например, звуци от стъпки).

По-сложни нива могат да съдържат допълнителни функции, като частици, визуална постобработка, стрийминг на нива и други.



Фигура 10: Мениджър на съдържание с две добавени нива

3.2 Софтуерната добавка Cesium for Unreal

Следващата част от настоящата дипломна работа ще запознае читателя със софтуерната добавка за създаване на терен Cesium for Unreal. Ще бъде разгледано как се генерира терен и как се трансформират реални координати, за да стане възможно позиционирането на обекти в игровия енджин. Ще разгледаме и някои от опциите за осветяване, които симулират реалната дневна светлина според зададени локация, дата и час.

3.2.1 Обща характеристика

Cesium for Unreal [9] съчетава 3D геопространствените възможности на Cesium с мощния енджин на Unreal Engine, позволявайки създаването на детайлни терени и 3D среди само с няколко кликвания. Това е безплатна софтуерна добавка за визуализация с отворен код, който предоставя пълномащабен и високоточен глобус по системата WGS84 (глобален стандарт за геодезически координати и триизмерна координатна система, използвана за определяне на местоположения на Земята.), вграден в Unreal

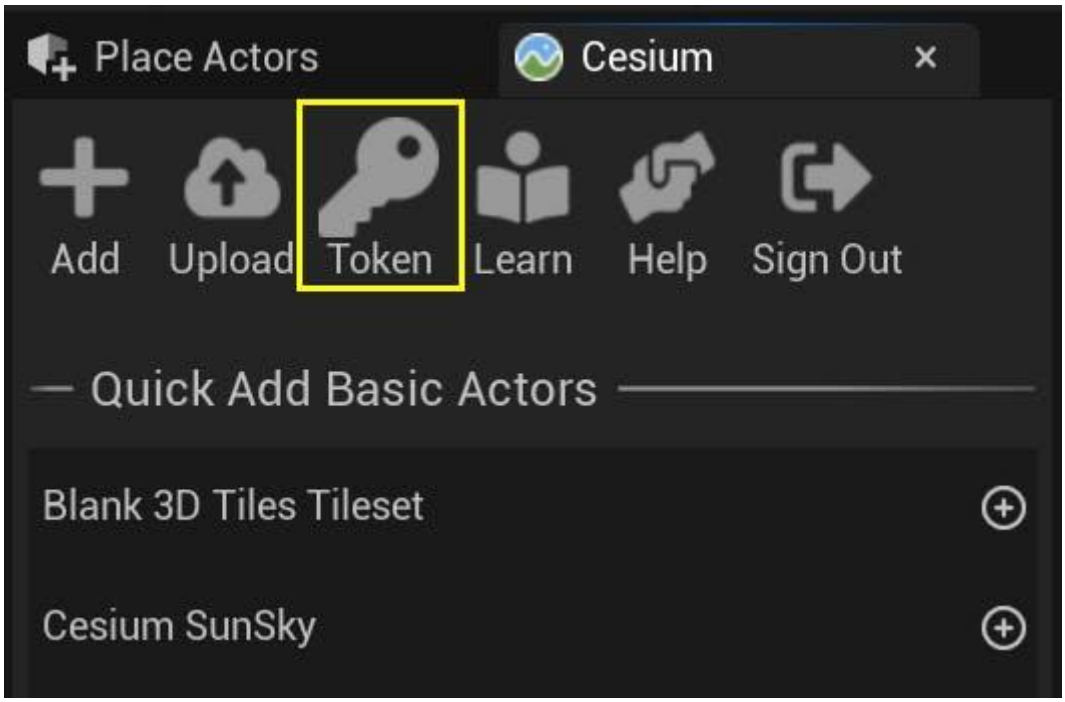
Engine. Софтуерната добавка осигурява визуализация в реално време на мащабна фотограметрия с висока разделителна способност и 3D съдържание във формат "3D Tiles" (Фигура 11). Интеграцията ѝ в Unreal Engine Editor е на много високо ниво, което означава, че разработчиците ще могат лесно и интуитивно да създават впечатляващи изживявания.



Фигура 11: Генериран терен с реални мащаби и 3D съдържание с помощта на Cesium for Unreal

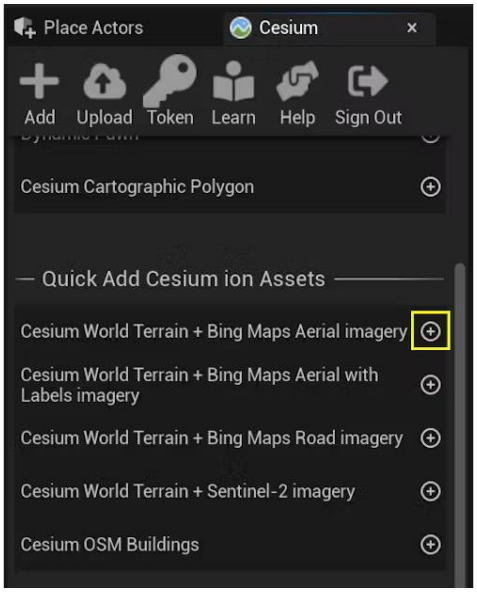
3.2.2 Генериране на терен

За да се използват функционалностите на софтуерната добавка, първо трябва да се активира от панела с всички софтуерни добавки. След това проектът трябва да се свърже с Cesium ion. От там ще се използват данните, необходими за генериране на терен. За целта е необходимо да се направи профил в Cesium ion и да се използва генерираният ключ в проекта (Фигура 12).



Фигура 12: Панелът за въвеждане на ключ за Cesium ion

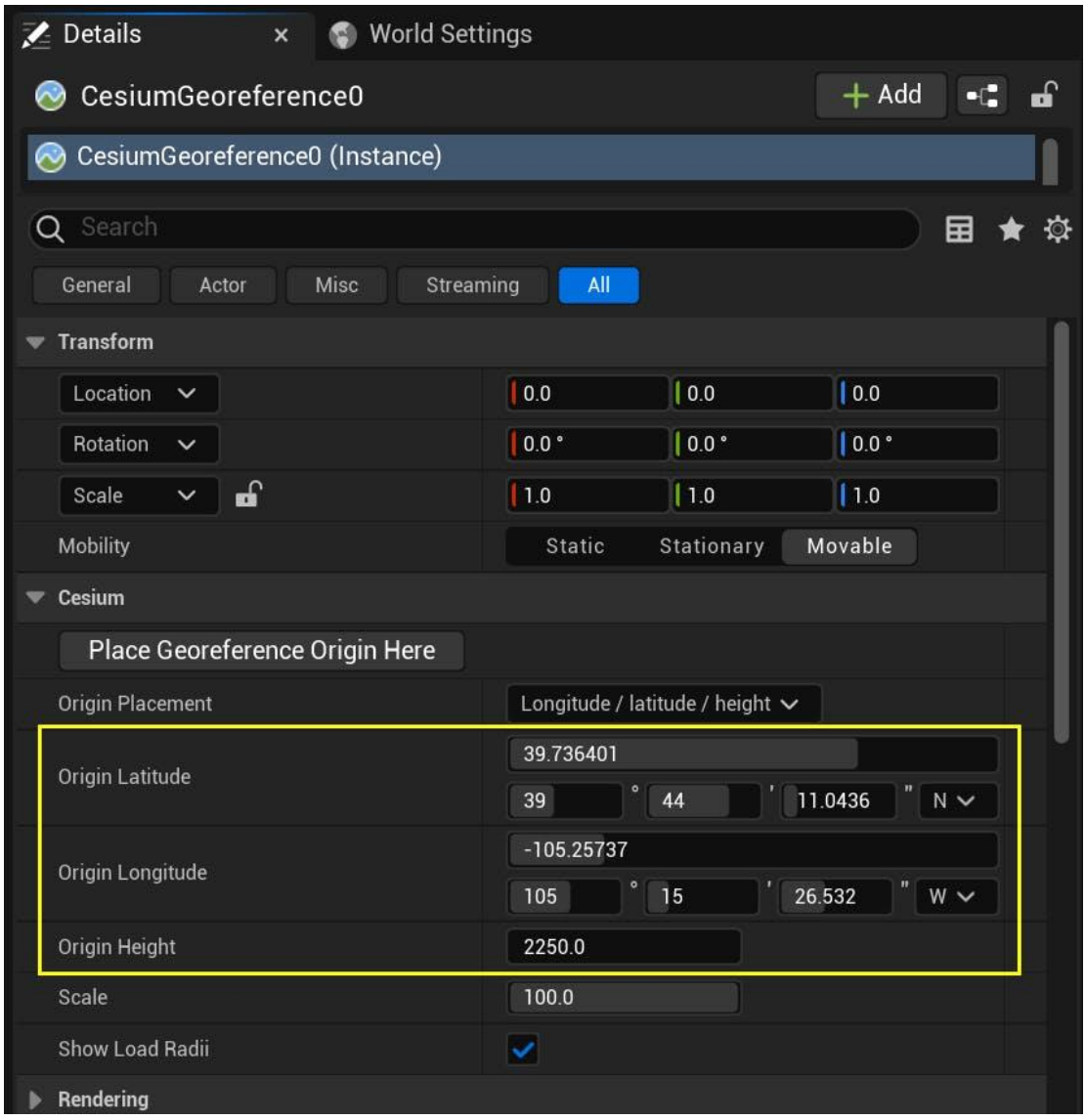
След като е въведен ключа, може да се генерира терен. За целта, от менюто на Cesium (Фигура 13) изберете опцията, която е най-подходяща за проекта и се натиска плюсът до нея.



Фигура 13: Прозореца Cesium

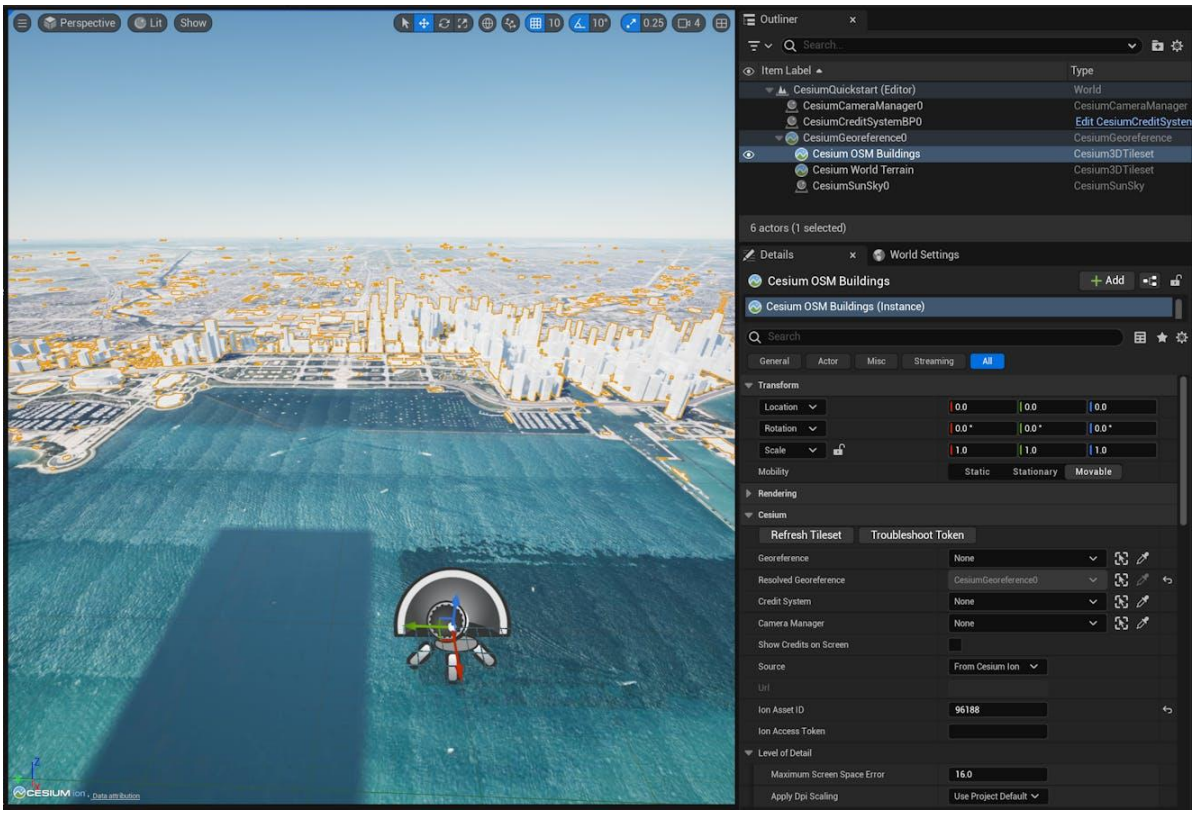
Това автоматично ще добави нови Cesium актьори към нивото. Един от тях – Cesium World Terrain – е наборът от плочки (tileset), който току-що беше създаден. Другите три – CesiumCameraManager, CesiumCreditSystemBP, и CesiumGeoreference – се

създават автоматично при първото добавяне на 3D Tileset или георефериран актьор към нивото. Освен тях, в нивото вече ще има и терен, който трябва да се настрои според нуждите на потребителя. В полето с детайлите (Фигура 14) можем да изберем местоположението на ниво, като въведем географските координати.



Фигура 14: Панел с детайли на Cesium терена

Cesium предоставя и възможност за добавяне на сгради благодарение на Cesium OSM Buildings actor. С него автоматично се генерират прости 3D модели на сградите, които се намират в нивото (Фигура 15).



Фигура 15: Сгради, генерирани с помощта на Cesium

3.2.3 Поставяне на обект в пространството

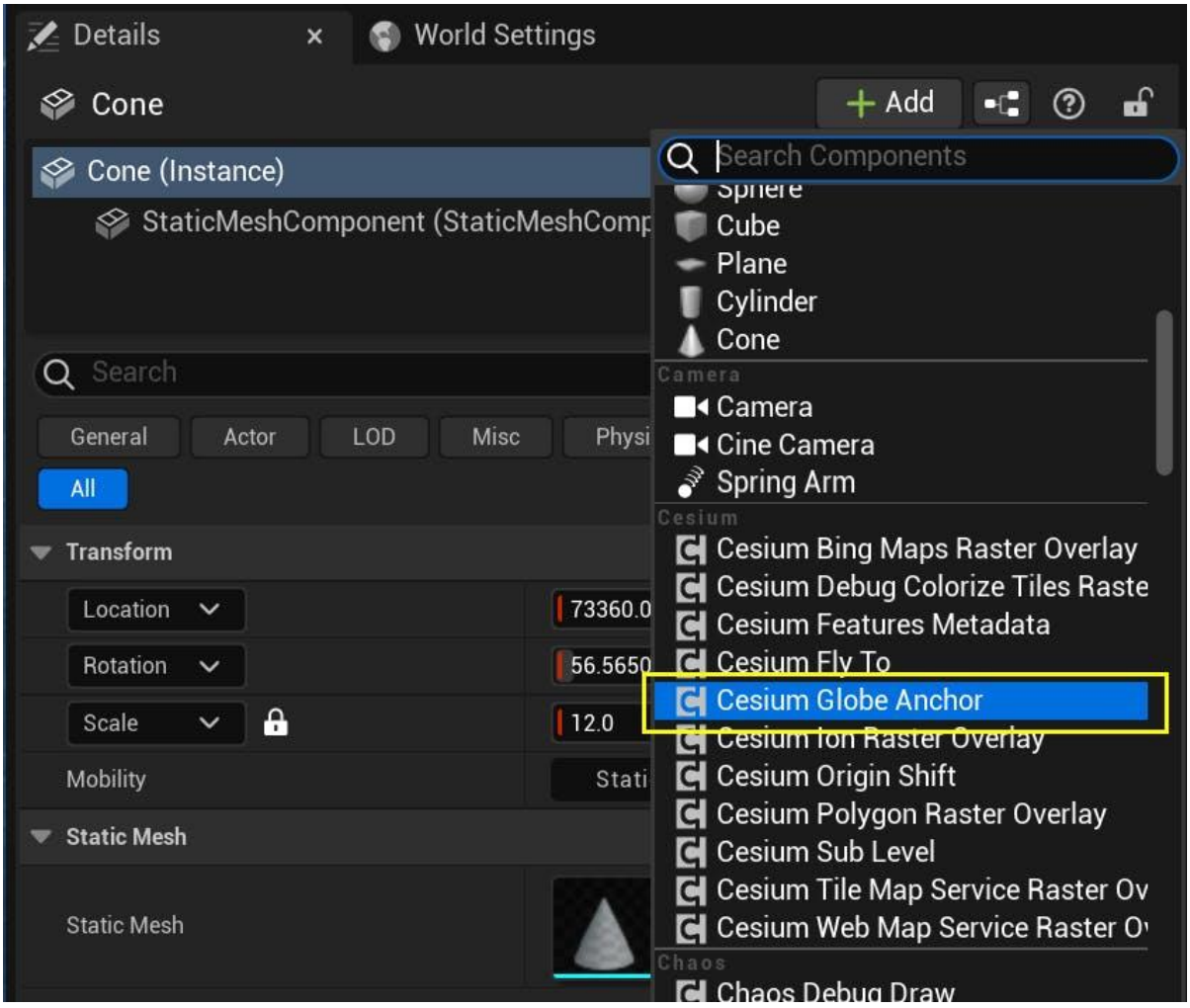
Когато обекти се поставят в Unreal Engine, им се задават конкретни координати в игровия свят. Връзката между тази координатна система на игровия свят и реалния свят обаче не е фиксирана. Тя се определя от параметъра на произхода на актьора CesiumGeoreference и може да бъде променена. Важно е, че когато произходът на CesiumGeoreference се промени, целият свят на Unreal Engine, включително и всички обекти в него, се премества на ново място върху глобуса.

Ако обект бъде поставен в планините близо до едно място и след това се промени произхода на CesiumGeoreference така че да сочи към друго място, обектът ще се премести към него. Затова обикновено е важно да не се променя произходът на CesiumGeoreference, след като е започнало изграждането на нивото.

Горепосоченото важи за почти всички обекти в Unreal Engine, но има някои обекти, които се подчиняват на различен набор от правила; наричаме тези обекти георефернтни обекти.

За разлика от обикновените обекти, които са фиксирани към координатната система на игровия свят, георефернтният обект е фиксиран към глобуса и има позиция, изразена с координати с двойна точност в система Earth-Centered, Earth-Fixed (ECEF) или, еквивалентно, чрез географска дължина, ширина и височина.

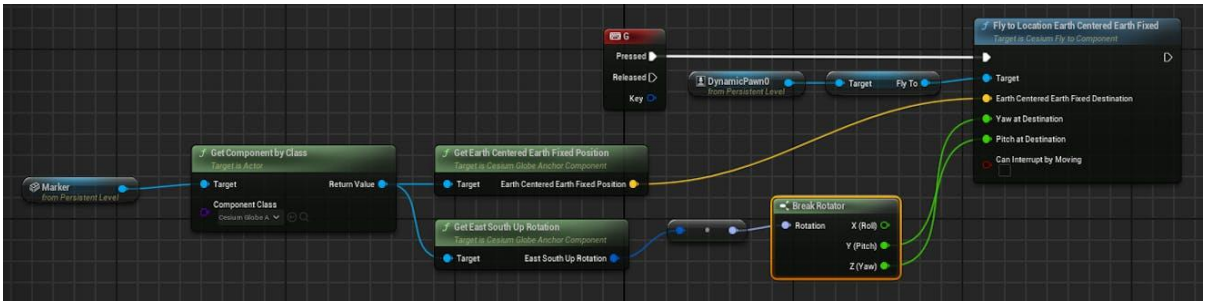
Ако тези координати го поставят на определено място, той ще остане там, докато не се премести ръчно от потребителя, дори ако се променят началните координати в CesiumGeoreference, така че нивото вече да е центрирано около напълно различен район. Cesium позволява превръщането на нормални обекти в георефериран обекти, чрез добавяне на CesiumGlobeAnchor компонент (Фигура 16).



Фигура 16: Превръщане на обект в георефериран обект

С помощта на този тип обекти и функциите за преобразуване на координати които предоставя Cesium е възможно динамично да се променя позицията и ориентацията на

обектите в свята (Фигура 17). Именно това е използвано за изработването на симулатора в настоящата дипломна работа.



Фигура 17: Летене на обект до предварително зададен маркер чрез функциите на Cesium

В исторически план, повечето игрови енджини използват координати с единична точност и плаваща запетая за своите векторни и матрични типове. Това създава проблем при приложения в глобален мащаб, тъй като планета с размерите на Земята изисква изключително големи стойности за координати, но също така и много прецизно позициониране с под-сантиметрова точност, когато се използват закръгляния (приближения). Ако това не се обработи правилно, се появява графичен артефакт, наречен „jitter“ – трептене или прескачане, при който грешките при закръгляне в големите стойности на координатите на камерата и близък обект водят до това обектът да изглежда сякаш "подскача", докато камерата или обектът се движат.

Глава 4. Реализация на приложението

В настоящата последна глава от същинската част на дипломната работа е разгледан процесът на създаване на софтуерната разработка. Разгледани са етапите, през които се премина, и проблемите, които се появиха по време на работата. Описано е и тяхното преодоляване. Освен това в детайли е разгледано как функционира симулаторът и какви технологии са използвани за разработката му.

Подобно на всички софтуерни разработки, тази разработка също премина през няколко различни етапа преди своето завършване. Съществуват множество утвърдени във времето модели за софтуерна разработка: линеен (например Waterfall или V-Mode) или цикличен (например Spiral или Agile).

Методът, използван за разработката на настоящото приложение за симулация на полет по данни от Mission Planner, е методът на бързо разработване на приложения. Този модел намалява времето и разходите за разработване, увеличава успеваемостта и се състои от следните четири фази:

- фаза на анализ и планиране
- фаза на проектиране
- фаза на разработка
- фаза на тестване

Приложението представлява визуална симулация на полет в реално време по данни, базирана на данни, дублиращи тези изпратени от дрона към наземния контрол чрез протокола MavLink. При стартирането му се зарежда камера от борда на дрона и околна среда в реален мащаб, генерирана с помощта на софтуерната добавка Cesium for Unreal. В бъдеще то може да бъде разширено с добавяне на симулации на физични сили. И възможност за представяне на предварително избрани постановки по време на полет, като внезапен порив на вятър. Както и с представяне на мисии, които вече са се провели, с помощта на запазени файлове от тях. В следващите параграфи подробно ще бъдат описани различните фази на процеса на разработване.

4.1 Фаза на анализ и планиране

Този етап се състои в изграждането на концепцията за приложението и анализирането на въпросите, свързани с него. Необходимо е да се проучат съществуващите технологии за разработка на приложението и да се избере най-подходящата технология. В конкретния случай се цели постигане на бързина и ефективност. Ето защо Unreal Engine 4 е технологията, която отговаря на тези две изисквания. Със своя удобен и интуитивен интерфейс, както и големия набор от инструменти и функционалности, процесът на разработка се улеснява. Нещо повече, с помощта на множество софтуерни добавки към Unreal Engine може да се създаде TCP връзка, чрез която лесно и бързо да получаваме данните. Също така с помощта на софтуерни добавки лесно може да се генерира терен за симулацията.

По време на тази фаза се свързахме и се срещнахме със специалист в областта на дроновете. Той ни запозна с процеса на предаване на данни от дрона до наземния контрол, протичането на мисията, сензорите и камерите, с които е оборудван стандартен дрон, както и други факти и идеи за създаването на симулатора. Освен това, на място видяхме самите дронове и терминалите от които се извършва наземният контрол.

4.2 Фаза на проектиране

Стандартно във фазата на проектиране се изготвя малък прототип, обмисля се структурата на самото приложение и се създава съпътстваща документация, която може да включва sequence диаграми, control flow диаграми, class диаграми, user диаграми и ръководство за експлоатация. За настоящото приложение е изготвена схема на потока на данните между различните компоненти от системата (Фигура 18). Данните се взимат от полета, като преминават през UDP сървър от който биват взети в Python скрипт който ги пакетира и изпраща към симулатора. След пристигането на данните в симулатора, те се записват в променливи, които се използват за интерполация на позицията на дрона в

игровата среда.

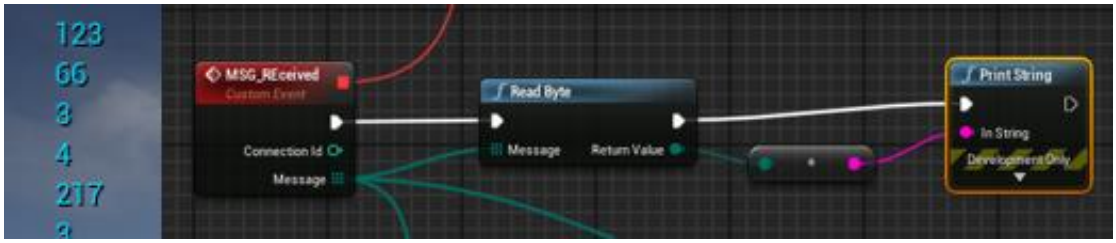


Фигура 18: Схема на потока на данни в разработката

4.3 Фаза на разработване

Във фазата на разработване се извършва програмиране и написване на код на базата на предоставените спецификации и проектирани модели. На този етап се очаква да се създаде работещо приложение, което може да има някакви проблеми за отстраняване, но основната част от работата е завършена.

За настоящата разработка започна с подбор на софтуерна добавка, която позволява превръщането на приложението в TCP клиент, за да е възможно извличането на данните от полета. За целта беше избран „TCP Socket Plugin“ [7] от голямото разнообразие от софтуерни добавки на сайта на Epic Games. Благодарение на него е изработена първата версия на симулацията. Тогава се появи и първият проблем. След инсталирането на софтуерната добавка и успешното ѝ свързване с Mission Planner (Фигура 19), осъзнахме че няма начин, по който да е възможно разделянето на пристигащите данни на използваеми променливи, като географска ширина, географска дължина и височина.



Фигура 19: Тест при получаване на битове от Mission Planner да се показват на екрана

За да се разреши този проблем, опитахме да използваме софтуерна добавка която да разделя пристигащите XML файлове, наречена „EasyXMLParser“ [8], но безуспешно.

След това беше предприет нов подход. Вместо да се изпращат всички данни директно до симулатора, решихме да ги изпращаме към отделен сървър написан на Python, където да се вземат само необходимите данни, и да се изпратят към симулатора.

За извличането на данните от Mission Planner към скрипта е използван разработеният от екипа на MavLink сървър „mavsdk“ [11], използващ UDP връзка. Той позволява лесно събиране на данни от полета в Python. След като данните вече са получени с помощта на този сървър, те се разделят на части. Тези части трябва да бъдат изпратени към симулатора, като е важно те да пристигнат цели по време на преноса. За целта частта от скрипта която се свързва със симулацията, играе ролята на TCP сървър. На няколко пъти се наложи промяна на първоначалния скрипт в процеса на разработката за да се разрешат различни проблеми. В следващата част от дипломната работа се описва подробно как работи финалният вариант на скрипта, както и неговите първоначални варианти, проблемите с тях и как са разрешени във финалната версия.

Първоначално скриптът отделяше и изпращаше само скоростта, височината и географските координати на полета. В блупринта трябваше да се записва сегашната позиция на дрона, а пристигащите данни да се задават като цел, към която трябва да отиде и чрез изчисления в blueprint-а да се зададе ротацията на обекта. И от сегашната точка до целта да се премине чрез интерполация. Но това не беше особено успешно. Поради тази причина беше допълнен скрипта за да може да изпраща данни за ориентацията на дрона в пространството Pitch, Yaw и Roll. Проблемът в тази втора версия идваше от закъсненията, с които пристигаха данните. Данните, отговарящи за позицията на дрона, пристигаха много по-често от тези, които отговарят за ориентацията. Което водеше до това, че дронът летеше, „гледайки“ в грешната посока за известно време, преди данните да се синхронизират. За отстраняването на този проблем, в последната версия на скрипта, данните от всички променливи се изчакват и пакетират, за да се изпратят заедно, с цел да се премахне възможността за десинхронизация.

4.3.1 Реализация на сървър с Python

4.3.1.1 Използвани библиотеки

За да се използват функционалности, които позволяват използването на TCP и UDP сървър чрез Python, и за да е възможна работата с данните на MavLink, са импортирани следните библиотеки от Фигура 20:

```
sdk_proxy.py
import asyncio
from mavsdk import System
import socket
import struct
```

Фигура 20: Списък на използваните библиотеки

Библиотеката Asyncio позволява асинхронно програмиране. Тоест, позволява изпълнението на няколко паралелни задачи, без да се блокира основният поток на програмата. Използва се, за да може получаването на данни от дрона и изпращането им към симулатора да са независими едно от друго, така че изчакването на нови данни да не блокира изпращането на предходните.

Подбиблиотеката System на библиотеката mavsdk ни позволява да работим с данните пристигащи от дрона, и да извличаме позицията и ориентацията му в пространството.

Библиотеката socket се използва за създаването и поддържането на TCP сървъра, чрез който се извършва изпращането на данни към симулатора.

Библиотеката struct позволява да пакетираме данните, които сме отделили в удобен за изпращане двоичен формат в нашия случай 4(float32) × 7 = 28 байта.

4.3.1.2 Функции

В скрипта са дефинирани три функции. Първата функция отговаря за приемането на данни от Mission Planner, втората – за изпращането им към симулатора, а третата, основна функция, стартира двата процеса, свързва се със сървъра mavsdk и създава TCP сървър.

Във функцията send_to_unreal() (Фигура 21) се изпълнява безкраен цикъл, в който данните се вземат от опашката и се пакетират в променлива data. След което се прави опит за изпращане към симулатора с помощта на .sendall(). Ако има грешка, в конзолата се визуализира съобщение с описание на грешката за по-лесно отстраняване. След опит за изпращане на информацията, независимо дали е успешен или не, се задава изчакване от 100ms, за да се контролира скоростта на изпращане. Тази функция има входен

параметър - връзката, към която се свързва симулацията. В конкретния случай това се случва на локалния адрес на машината, която извършва заявката: „127.0.0.1:5005“.

```
async def send_to_unreal(conn):
    while True:
        lat, lon, alt, speed, roll, pitch, yaw = await telemetry_queue.get()
        data = struct.pack('fffffff', lat, lon, alt, speed, roll, pitch, yaw)
        try:
            conn.sendall(data)
        except Exception as e:
            print(f"[ERROR] Failed to send data to Unreal: {e}")
            break
        await asyncio.sleep(0.1)
```

Фигура 21: Функцията която изпраща данните към симулатора

Във функцията collect_telemetry() (Фигура 22) се събират данните от полета, разделят се само тези, които ще се изпращат, и се запазват в опашката, от която се извличат от функцията send_to_unral(). За да се случи това, първо се проверява дали има налична връзка с дрон. След като потвърдим, че има налична връзка с дрон и има налични данни, започва цикъл, който извлича тези данни, които са ни нужни, и ги запазва в променливи с говорещи имена (lat, lon, alt, speed, pitch, roll, yaw). След като се съберат всички променливи, те се запазват в един общ пакет, като се отделят по 4 байта за всяка променлива, или общо 28 байта. Тази променлива се добавя в опашката, за да се изпрати, след което отново е поставено забавяне за контрол на потока при необходимост. В случая то е зададено на 50 ms.

```python
async def collect_telemetry(drone):
    async for position in drone.telemetry.position():
        latest_position = position
        break

    async for velocity in drone.telemetry.velocity_ned():
        latest_velocity = velocity
        break

    async for attitude in drone.telemetry.attitude_euler():
        latest_attitude = attitude
        break

    print("Telemetry streams confirmed, starting loop...")

    # Start streaming telemetry
    async for position in drone.telemetry.position():
        velocity = await drone.telemetry.velocity_ned().__anext__()
        attitude = await drone.telemetry.attitude_euler().__anext__()

        lat = position.latitude_deg
        lon = position.longitude_deg
        alt = position.absolute_altitude_m

        vx = velocity.north_m_s
        vy = velocity.east_m_s
        vz = velocity.down_m_s
        speed = (vx**2 + vy**2 + vz**2)**0.5

        roll = attitude.roll_deg
        pitch = attitude.pitch_deg
        yaw = attitude.yaw_deg

        print(f"Send: lat={lat:.6f}, lon={lon:.6f}, alt={alt:.2f}, spd={speed:.2f}, rpy={roll:.1f}/{pitch:.1f}/{yaw:.1f}")
        await telemetry_queue.put((lat, lon, alt, speed, roll, pitch, yaw))

        await asyncio.sleep(0.05)
```

Фигура 22: Функцията която се грижи за получаването на данните

Основната функция main()(Фигура 23), от която започва скриптът, се занимава със свързването към сървъра mavsdk, от който се вземат пристигащите данни. Създава и TCP сървър, към който симулаторът се свързва, за да получи данните. След това започват два асинхронни процеса: събиране на данни и тяхното изпращане.

```python
async def main():
    print(f"Listening for Unreal on {UNREAL_HOST}:{UNREAL_PORT}...")
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.bind((UNREAL_HOST, UNREAL_PORT))
    server_socket.listen(1)

    conn, addr = server_socket.accept()
    print(f"Unreal connected from {addr}")

    drone = System(mavsdk_server_address='localhost', port=50051)
    await drone.connect(system_address="udp://:14560")

    print("Waiting for drone...")
    async for state in drone.core.connection_state():
        if state.is_connected:
            print("Drone discovered!")
            break

    await asyncio.gather(
        collect_telemetry(drone),
        send_to_unreal(conn)
    )
```

Фигура 23: Функцията main()

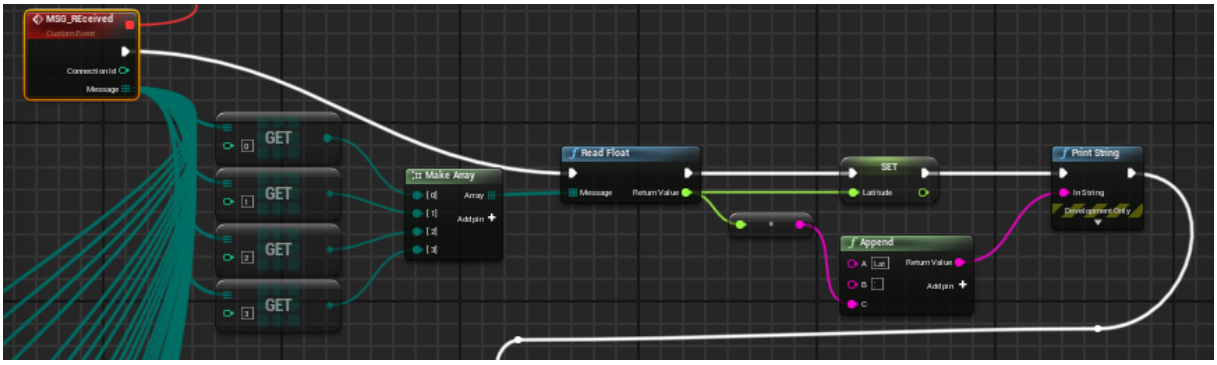
4.3.1.3 Изпълнение на скрипта

За да може скриптът да започне работа е необходимо да се изпълнят две команди в два терминала. Първата команда „.\mavsdk_server.exe -p 50051 udpin://0.0.0.0:14560“ стартира mavsdk сървъра, който трябва да е предварително свален. Той се свързва с Mission Planner (който предварително трябва да сме конфигурирали да изпраща MAVLink на порт 14560) и позволява на Python да използва MAVSDK API на порт 50051, за да е възможно извличането на данните. Във втория терминал след изпълнение на командата „python mavsdk_proxy.py“ се стартира скриптът. Изчаква се Unreal да се свърже. След като връзката бъде установена, данните започват да се изпращат през TCP към Unreal.

След като процесът на пренос на данни от MavLink към симулатора беше разгледан, следва да се обърне внимание на начина, по който тези данни се използват за визуализация на полета.

4.3.2 Визуализиране на полета в Unreal Engine

Знаем, че всеки пакет от данни се състои от 7 променливи, като за всяка една от тях са заделени по 4 байта. Затова, в блупринта, всеки път, когато се получат данни те се разделят и записват в променливи с говорещи имена. На Фигура 24 е показано как изглежда отделянето и запазването на географската ширина, на която се намира дронът. За другите променливи е абсолютно аналогично, като се променят номерата на байтовете, които се използват, и променливите, в които се запазват.



Фигура 24: Записване на пристигащата географска ширина в променлива

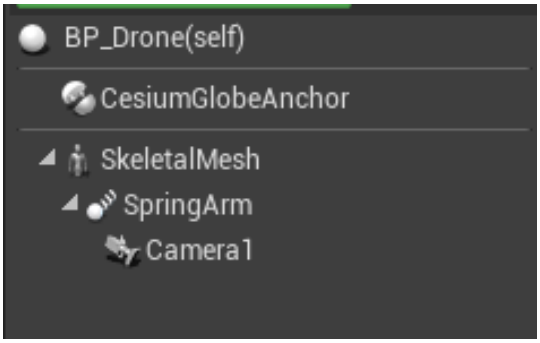
След като всички данни, необходими за симулацията, са налични, подбрахме модел на дрон за неговата визуализация. Използван е безплатният модел на UAV MQ-9 Reaper West (Фигура 25) от сайта на Vigilante [10].



Фигура 25: Модел на дрона

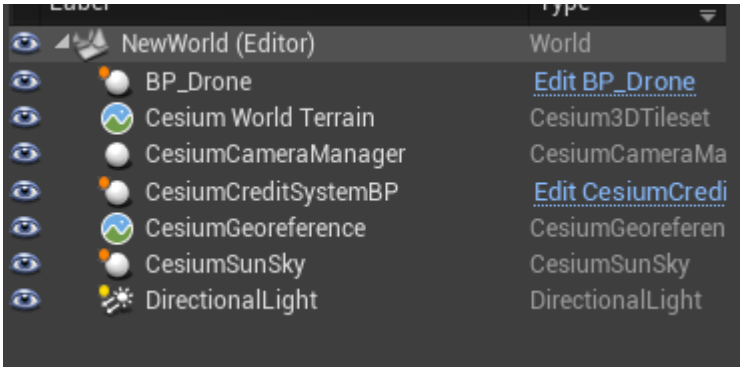
Skeletal mesh моделът е сложен като начален компонент на блупринта, от който се извличат данните, и е добавен към нивото. За да е възможно да се използва камера е добавен камера компонент, който е свързан със skeletal mesh-а благодарение на spring arm компонент. Освен това, на по-късен етап се наложи да добавим и компонент CesiumGlobeAnchor, за да е възможно преместването на дрона в пространството със

функциите, вградени в софтуерната добавка Cesium for Unreal. На Фигура 26 е показана е финалната структура на блупринта на дрона.



Фигура 26: Структура на блупринта

След като блупринтът е конфигуриран, остава само да се направи нивото, на което ще се визуализира полетът. За целта е използвана софтуерната добавка Cesium for Unreal. Създадено е ново ниво, към което е добавен терен с помощта на сателитни изображения от базата данни на Cesium. Това автоматично добавя Cesium World Terrain, CesiumCameraManager, CesiumGeoreference и CesiumCreditSystemBP към ниво. За да е осветено нивото е добавен и CesiumSunSky. Чрез задаване на дата и час се симулира точната дневна светлина на локацията, в която се намираме. След това ниво има следния изглед от Фигура 27.

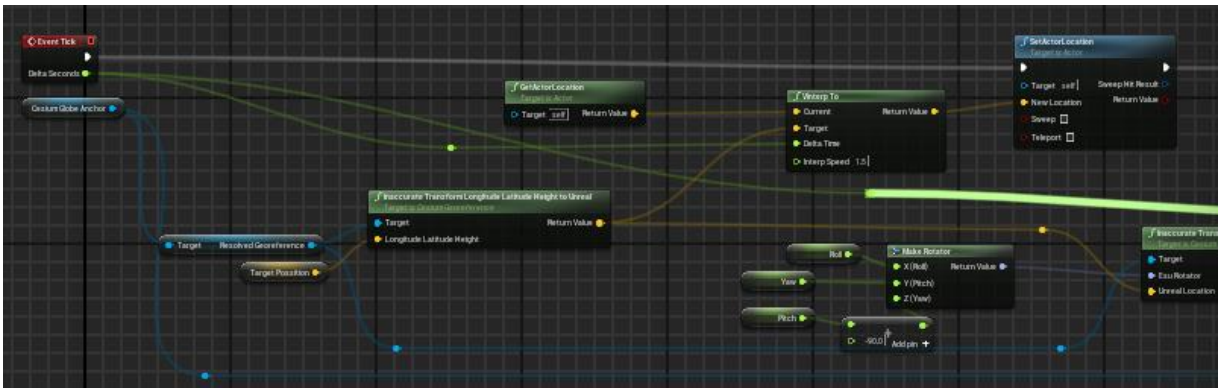


Фигура 27: World outliner

След като теренът е добавен терен е необходимо да бъде конфигуриран от панела с детайлите, за да бъде позициониран на правилното място от света, където ще започне мисия. За целта се вземат координатите на началната точка на мисията, („Home location“), и се поставят в полето с детайли на CesiumGeoreference. Това ще премести сцената на тези координати. Не би трябвало да има проблем, ако не зададем тази точка

понеже по време на симулацията се гледаме от движещата се камера на борда на дрона, но е по-добре да се подсигурим, а и това улеснява процеса на разработка и откриването на проблеми с визуализацията.

С набавените данни и създадения терен, остава само да се изпълни визуализацията на полета. Фигура 28 показва как се извършва движението в пространството. При всяко обновяване на играта се взема текущата позиция на дрона в пространството и се извършва интерполация между нея и последните данни, получени от Mission Planner, след което с получения вектор задаваме нова позиция на дрона. Аналогично се извършва и ротацията с единствената разлика, че преди да извършим интерполация изваждаме 90 градуса от променливата Pitch, тъй като MavLink и Unreal използват две различни системи за задаване на ротация в пространството и е необходимо да да компенсираме разминаването.



Фигура 28: Визуализация на движение в пространството

4.4 Фаза на тестване

В този етап от софтуерната разработка се правят различни видове тестове, като integration tests, unit tests, functional tests, performance tests, system tests или smoke tests, за да се открият и отстранят различни проблеми, които са останали след фазата на разработване. След края на тази фаза се очаква да има завършено и работещо приложение с пълна функционалност. Към тази фаза може да бъде добавена и фаза на поддръжка, която следва да бъде допълнително уточнена между разработчиците и потребителите на софтуера. В текущата разработка са правени само ръчни тестове. След завършване на някоя от функционалностите например след получаване на данните, са правени малки тестове, за да се потвърди, че всичко е наред. За по-големи разработки

приложението може да се предостави на други лица, които да тестване, тъй като така е по-вероятно да бъде открит проблем.

Заключение

В дипломната работа са разгледани възможностите за управление на безпилотни летателни апарати. Разгледани са различни методи за пренос на данни в интернет, както и техните предимства и недостатъци. Разгледани са възможностите за създаване на игри и виртуални изживявания с помощта на Unreal Engine и множеството налични софтуерни добавки към него.

След проучване, именно тези технологии бяха избрани като най-подходящи за разработване на симулация в реално време, която може да се използва за прототип върху който в бъдеще да стъпи подобна по-голяма софтуерна разработка. Разработката акцентира върху анализа на поставената задача, подходите за нейното решаване, програмната реализация и крайните резултати от работата на предложения алгоритъм. Изработеният софтуер предоставя модул, който отговаря за трансфера на данни от наземния контрол към изработения симулатор, в който се визуализира в реално време полетът. За визуализацията се използва интерполация на настоящата позиция на автономния дрон и пристигащата позиция от наземния контрол.

Авторска справка за приносите

Авторът е допринесъл за настоящата дипломна работа, разработването на приложение за симулиране на полет на безпилотен летателен апарат използвайки данни от автопилота на Mission Planner и Unreal Engine 4. Крайният резултат е постигнат чрез:

- изготвяне на концепция за приложението
- проучване, анализ и избор на подходящата технология за разработка
- проектиране с помощта на изработен прототип
- създаване на подходящ дизайн
- имплементиране на програмната логика

Перспективи за бъдещо развитие

Следните стъпки са предвидени за бъдещото развитие на разработената физично-базирана симулация на полет по данни от автопилота на Mission Planner:

- Добавяне на възможност за програмиране на непредвидени ситуации по време на полета, като внезапен порив на вятъра
- Добавяне на възможност за визуализация на полет с помощта на логове от вече проведена мисия
- Добавяне на възможност за визуализиране на повече от един полет (С по-нисък приоритет)
- Събиране на обратна връзка от евентуални потребители
- Създаване на проект с отворен код и добавяне на сътрудници

Библиография

[1] Документация на автопилота Ardupilot, достъпна на 14.06.2025, от: https://ardupilot.org/ardupilot/

[2] Документация на софтуера за наземен контрол Mission Planner, достъпна на 14.06.2025, от: https://ardupilot.org/planner/docs/mission-planner-overview.html

[3] Документация на специализирания протокол за пренос на данни MavLink, достъпна на 14.06.2025, от: https://mavlink.io/en/about/overview.html

[4] Документация към Unreal Engine за работа с нивата, достъпна на 14.06.2025, от: https://dev.epicgames.com/documentation/en-us/unreal-engine/managing-multiple-levels-in-unreal-engine

[5] Документация към Unreal Engine за работа с blueprint-и, достъпна на 14.06.2025, от: https://dev.epicgames.com/documentation/en-us/unreal-engine/blueprints-visual-scripting-in-unreal-engine

[6] Документация към Unreal Engine за работа със софтуерни добавки, достъпна на 14.06.2025, от: https://dev.epicgames.com/documentation/en-us/unreal-engine/working-with-plugins-in-unreal-engine

[7] Софтуерна добавка за приемане на данни чрез TCP в Unreal проект, достъпна на 14.06.2025, от: https://www.fab.com/listings/48db4522-8a05-4b91-bcf8-4217a698339b

[8] Софтуерна добавка за отделяне на части от XML файл в Unreal проект, достъпна на 14.06.2025, от: https://www.fab.com/listings/6bb95d43-7c38-4609-9df2-064fd32b8abf

[9] Документация на софтуерната добавка Cesium, достъпна на 14.06.2025, от: https://cesium.com/learn/unreal/unreal-photorealistic-3d-tiles/

[10] Модел на дрон използван, за визуализация, достъпен на 14.06.2025, от: https://marketplace.vigilante.us/product/uav-mq-9-reaper-west/

[11] MAVSDK сървър използван за взимане на данни от Mission Planner и позволяващ използването им в скрипта, достъпен на 14.06.2025, от: https://github.com/mavlink/MAVSDK/releases

[12] Orogun, A. O. (n.d.). Brief insight into Transmission Control Protocol (TCP). ResearchGate. Достъпна на 14.06.2025, от: https://www.researchgate.net/publication/329337483_Brief_insight_into_transmission_control_protocol_TCP

[13] Saide, M. (2024, April 25). Exploring the TCP/IP protocol suite: Architecture, dominance, and future challenges in data communication. SSRN. https://ssrn.com/abstract=4885418

[14] Malinowski, A., & Wilamowski, B. M. (2011). User Datagram Protocol (UDP). In Industrial Communication Systems (pp. 59–1 to 59–10). Auburn University. Достъпна на 14.06.2025, от: https://www.eng.auburn.edu/~wilambm/pap/2011/K10148_C059.pdf

[15] Fortinet. (n.d.). What is User Datagram Protocol (UDP)?. Достъпна на 14.06. 2025, от: https://www.fortinet.com/resources/cyberglossary/user-datagram-protocol-udp

[16] Платформата на DYIDrones достъпна на 14.06.2025, от: https://diydrones.com/

[17] ClubZ. (2024). Украйна показа кадри от унищожението на руски Су-57. Достъпна на 16.06.2025, от: https://clubz.bg/162380